

한경찬

데이터에서 의사결정 근거를 만드는 데이터 분석가.

EDA 설계부터 모델 해석까지, 분석의 모든 판단을 수치로 정당화합니다.

Tiger.inc — Hybrid AI 법인카드 정산 자동화

이젠, 안심 — 청년 지원 RAG 에이전트 · Django 백엔드 리빌드

TrendIt — YouTube 트렌딩 지속성 예측 플랫폼

Forecasting BTC/ETH — Markov Regime-Switching HAR 학부 연구 논문

hkc3511@naver.com

blog.naver.com/hkc3511

About

분석 방향을 설계하고 실제 의사결정에 필요한 근거를 수립하는 데이터 분석가입니다. 실무 프로젝트에서 데이터 전처리와 누수 방지 원칙 수립, 모델링·실험 구조 기획을 직접 주도했습니다.

AI Agent와 LLM 기반 시스템을 설계할 때는 "모델이 틀릴 수 있다"는 전제에서 출발합니다. 최종 판단이 필요한 지점은 검증 가능한 로직에 맡기고, LLM은 설명이나 검색처럼 보조적인 역할만 담당하도록 역할을 나눠 신뢰할 수 있는 구조를 만듭니다.

분석 설계가 중심이지만, 필요하면 백엔드 서비스 레이어 설계·인증·DB 구축·배포 환경 연동까지 직접 처리할 수 있는 엔지니어링 실행력도 갖추고 있습니다.

ML / Modeling	XGBoost · LightGBM · scikit-learn · PyTorch · NLP · LLM · RAG
Data Engineering	pandas · NumPy · SQL · MySQL · PostgreSQL
Visualization	matplotlib · seaborn · Plotly · Dashboard
Engineering	FastAPI · Django · Streamlit · React · Git · LangGraph · Neo4j · Docker · AWS

Tiger.inc

Hybrid AI 법인카드 경비 정산 자동화 플랫폼

팀 5인 · 데이터 전처리 · ML 이상탐지 설계 · AI Agent 개발 · 문서 정합성 관리

97.3%

운영 자동처리율

0.5865

이상탐지 PR-AUC

0.906

RAG 검색 MRR

승인·반려 판정은 언제나 결정론적 Rule Engine이 내립니다. AI가 개별 정산 건의 최종 판정을 내리는 경로는 시스템 어디에도 없습니다.

회계 담당자가 영수증을 보며 내규를 일일이 뒤져 승인·반려를 정하는 반복 노동을 줄이는 정산 플랫폼입니다. 규칙(결정론)·통계(ML)·생성형 AI가 각각 다른 역할을 맡되, 확정
은 반드시 사람이 합니다.

담당 — 195만 건 카드 거래 데이터 전처리, 비지도 이상탐지 모델 설계, Draft·Rule·Risk Review 3개 AI 보조 파이프라인 개발, 프로젝트 문서 정합성 검토. Agent마다
"AI가 낼 수 없는 값"을 막는 방식이 다릅니다 — Draft는 출력 스키마 제한, Rule은 액션 화이트리스트, Risk Review는 근거 대조 검증. **프롬프트 지시가 아니라 구조로 경계
를 강제한다**는 것이 셋을 관통하는 원칙입니다.

핵심 설계 판단

— 가상기업을 설계해 검증 가능한 문제로 바꾸다

실제 기업의 정산 데이터와 내규는 공개되지 않습니다. 가상기업 "타이거 주식회사"의 조직도·직급 체계부터 만들고 사내 규정 6종·법령 3종·중빙 서식 8종을 파생시켜 검증 가능한 문
제로 재구성했습니다. 인가는 역할이 아니라 6개 Capability 단위로 설계해, 룰을 만드는 사람과 활성화하는 사람을 분리했습니다.

— Rule Agent가 낼 수 있는 최악의 결과는 "한 번 더 검토"

LLM이 만들 수 있는 액션은 REJECT·RETURN·REVIEW뿐이고 출력 스키마에 PASS 자체가 없습니다. 룰을 아무리 더해도 통과가 늘어날 수 없고 줄어들기만 합니다. 조건식도 임의
코드 대신 몇 가지 연산자만 조합하는 전용 DSL로 제한해 코드 실행 가능성을 원천 차단했습니다.

— 라벨 0건 콜드스타트 — 11종 비교와 통계 검증으로 확정

배포 전에는 승인/반려 기록이 0건이고, 카드사 부정사용 라벨은 회사 내규와 정의가 달라 정답으로 쓸 수 없었습니다. 라벨 없이 쓸 수 있는 비지도 11종을 전부 실측 비교해 Isolation
Forest를 채택했고(PR-AUC 0.5865, 2위 COPOD 0.5330), 한 번의 채점을 믿지 않고 5-fold 교차검증에 paired t-test까지 적용해 유의성을 확인했습니다($t=4.929$, $p=0.0079$).

— 비율 목표가 아니라 위험 밀도로 3단계를 나누다

상위 N%를 잘라 LLM 호출을 줄이면 애매한 건이 하위 구간에 섞여 LLM 0회로 새어나갑니다. 그래서 위험 밀도가 실제로 갈리는 지점(score 0.011·0.072)을 경계로 삼았습니다. 다
만 이 분리점은 위험 라벨 5건이라는 작은 표본에서 나온 값이라, 코드 주석에도 "분리점이 아니라 힌트"라고 명시했습니다.

— Agent끼리 직접 부르지 않는다 — Blackboard 패턴

3개 Agent는 서로를 호출하지 않고 Postgres와 Chroma라는 공용 저장소에 결과를 남깁니다. 증빙 추출 Agent가 읽어낸 확인 인원이 EvalContext에 올라가면, Rule Engine은 그
값으로 1인당 한도를 계산하고 Risk Review는 신고 인원과의 차이를 봅니다. 신고값과 확인값을 애초에 다른 필드로 분리해, 같은 값을 두고 다투는 문제 자체를 없앴습니다.

— 규정 문서는 조(條) 단위로 자른다

문단 단위로 임의로 자르면 서로 다른 조문에 한 조각에 섞여 엉뚱한 인용이 발생합니다. 검색은 작은 조 단위로 하되 LLM에 넘길 때는 조 전체를 붙여주는 Parent-Child 방식을 썼습
니다. 긴 조문에서 벡터가 평균화되는 것을 막기 위해 parent는 앞 400자만 임베딩하고 표는 별도 청크로 분리했습니다.

— 룰이 정교해질수록 지표가 나빠지는 함정을 바로잡다

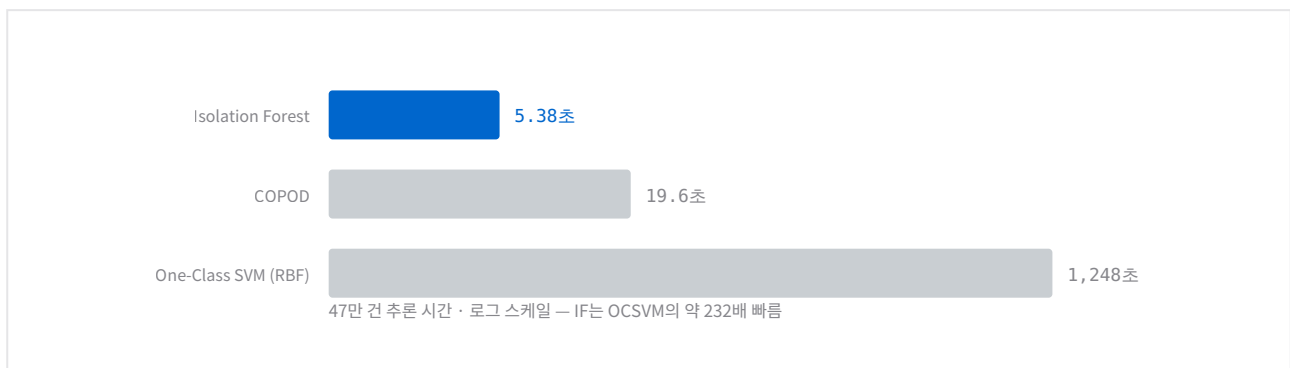
자동처리율을 PASS 비율로 정의하면, 사람이 다시 봐야 하는 REVIEW·RETURN이 하나만 늘어도 지표가 떨어집니다. 룰을 제대로 만들수록 성적표가 나빠지는 셈입니다. "1 - (검토
필요 건 / 전체)"로 정의를 다시 잡아, 룰이 정교해질수록 지표가 실제로 개선되도록 맞췄습니다. 지표는 성과를 자랑하는 도구가 아니라 개선 방향을 가리키는 도구여야 한다는 게 이
수정의 이유입니다.

시스템 흐름 — 판정 시점과 두 개의 별도 트랙



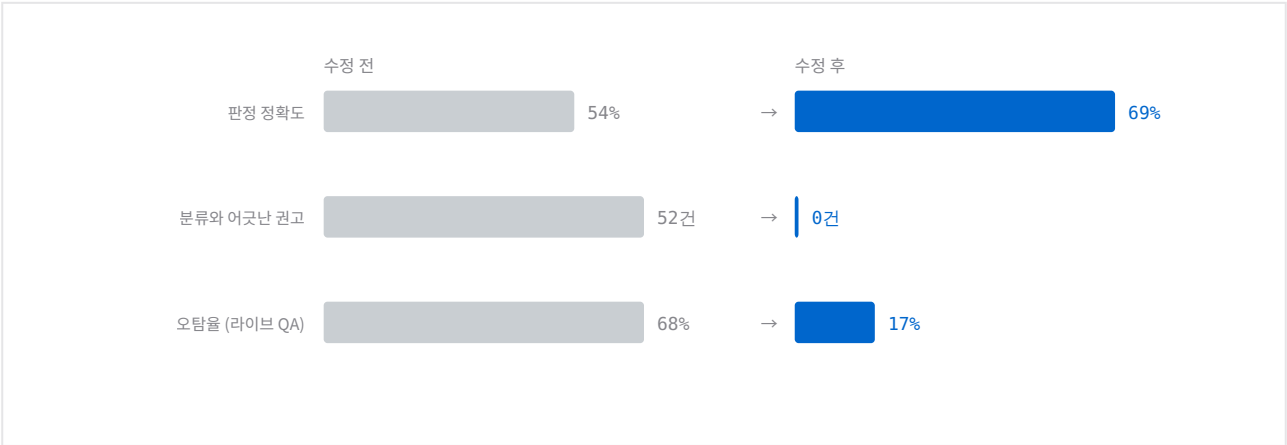
판정은 "팀 취합" 시점에 확정되고 회계 제출은 그 결과를 재사용합니다. 엔진의 REJECT조차 재제출 가능한 RETURNED이며, 되돌릴 수 없는 최종반려는 회계 담당자가 직접 누르는 REJECT뿐입니다. 오른쪽 트랙의 룰 활성화는 회계팀장만 할 수 있어 두 트랙의 최종 결정자가 다릅니다.

추론 속도 — 정확도만으로 고른 것이 아니다



Isolation Forest는 148만 건 전체를 쓰고도 47만 건 추론에 5.38초가 걸립니다. One-Class SVM은 5만 건만 서버샘플링하고도 1,248초(약 21분) — 정확도가 낮으면서 느리기까지 해 실운영 후보에서 제외했습니다.

Risk Review Agent — 결함 2건 수정 전후



E2E 검증셋 100건 재평가 결과입니다. 분류와 어긋난 권고가 52건에서 0건으로 사라진 것이 정확도 상승보다 중요한 변화입니다. 남은 미탐 25건의 원인은 전달 누락·관용어 인식 한계·별표 값 미참조 3가지로 전부 특정해렸습니다.

정량 성과

97.3%

운영 자동처리율 seed_adopted 185건 · 3개월 시뮬레이션

실제 상태머신을 거친 정산 185건 중 사람이 다시 안 봐도 되는 비율. 남은 검토 5건은 전부 규칙이 의도적으로 확정하지 않는 청탁금지법 리스크입니다.

0.5865

이상탐지 PR-AUC 비지도 11종 중 1위 · $p=0.0079$

2위 COPOD(0.5330) 대비 우위가 5-fold paired t-test에서 통계적으로 유의했습니다. 라벨이 없는 상태에서 고른 모델이라 절대 성능보다 상대 비교와 검정 절차가 근거입니다.

MRR 0.906

RAG 검색 정확도 Recall@5 0.950

파싱 89.3점·청킹 98.5점까지 단계별로 나눠 채점했고, 이 검증 과정에서 두 Agent가 공유하던 검색 스코프 누수 결함을 발견·수정했습니다(scope 정합 100/100 재검증).

기술 스택

Python · FastAPI · FastMCP 2.4 · OpenAI SDK · Chroma · text-embedding-3-large · docling · scikit-learn · Isolation Forest · Django 5.1 + DRF · PostgreSQL 16 · Docker Compose

github.com/SKNETWORKS-FAMILY-AICAMP/SKN29-FINAL-1TEAM

이젠, 안심

청년 지원 통합 탐색 에이전트 — Django 리뉴얼·실배포

팀 5인 · Backend Core Lead (Phase 2) · Backend 설계 참여 (Phase 1)

6개

도메인 앱 구조

37/37

기능 테스트 Pass

EC2

배포 환경 연동

프로토타입을 실제로 운영 가능한 서비스 구조로 다시 세우는 것이 이 프로젝트에서 맡은 일이었습니다.

은통청년·K-Startup·고용24 공공 API로 흩어진 청년 지원 정보를 통합해 AI 챗봇으로 맞춤 정책을 추천하는 서비스입니다.

Phase 1 — FastAPI + LangGraph 기반 RAG 프로토타입에서 서비스 레이어 설계와 Neo4j Hybrid RAG 구현을 담당.

Phase 2 — Django REST Framework로 백엔드 전체를 리빌드하며 인증·정책·커뮤니티·알림·업로드 전 도메인 앱 구조 설계와 리딩을 맡았습니다. AWS RDS PostgreSQL로 서비스 DB를 직접 구축하고, EC2에 접속해 Docker 환경에서 백엔드·프론트를 연동·검증했습니다.

서버 프로비저닝은 팀 내 인프라 담당자가, AI 챗봇 RAG 파이프라인 이식은 AI Backend 담당자가 진행했습니다.

핵심 설계 판단

— 단일 서비스 레이어를 6개 도메인 앱으로 재설계

Phase 1의 FastAPI 프로토타입은 서비스 레이어 하나에 모든 로직이 모여 있었습니다. Phase 2에서 users·policies·community·notifications·uploads·common으로 도메인을 분리해 각 앱이 독립적으로 배포·테스트 가능하도록 경계를 나눴고, 팀 내 Backend Core 담당으로 전체 API 설계를 주도했습니다.

— 인증 실패가 서비스 전체를 막지 않도록

Simple JWT에 refresh token rotation을 적용하되, 만료·손상된 토큰이 있어도 로그인 같은 AllowAny 엔드포인트는 통과하도록 Optional JWT 인증을 설계했습니다. "정상 케이스"보다 "예외 케이스"가 중요한 로직이라, 만료·손상 토큰 상황을 직접 재현해 의도대로 막히지 않는지 확인했습니다.

— "실제로 켜졌을 때 필요한 것"을 하나의 기준으로 묶다

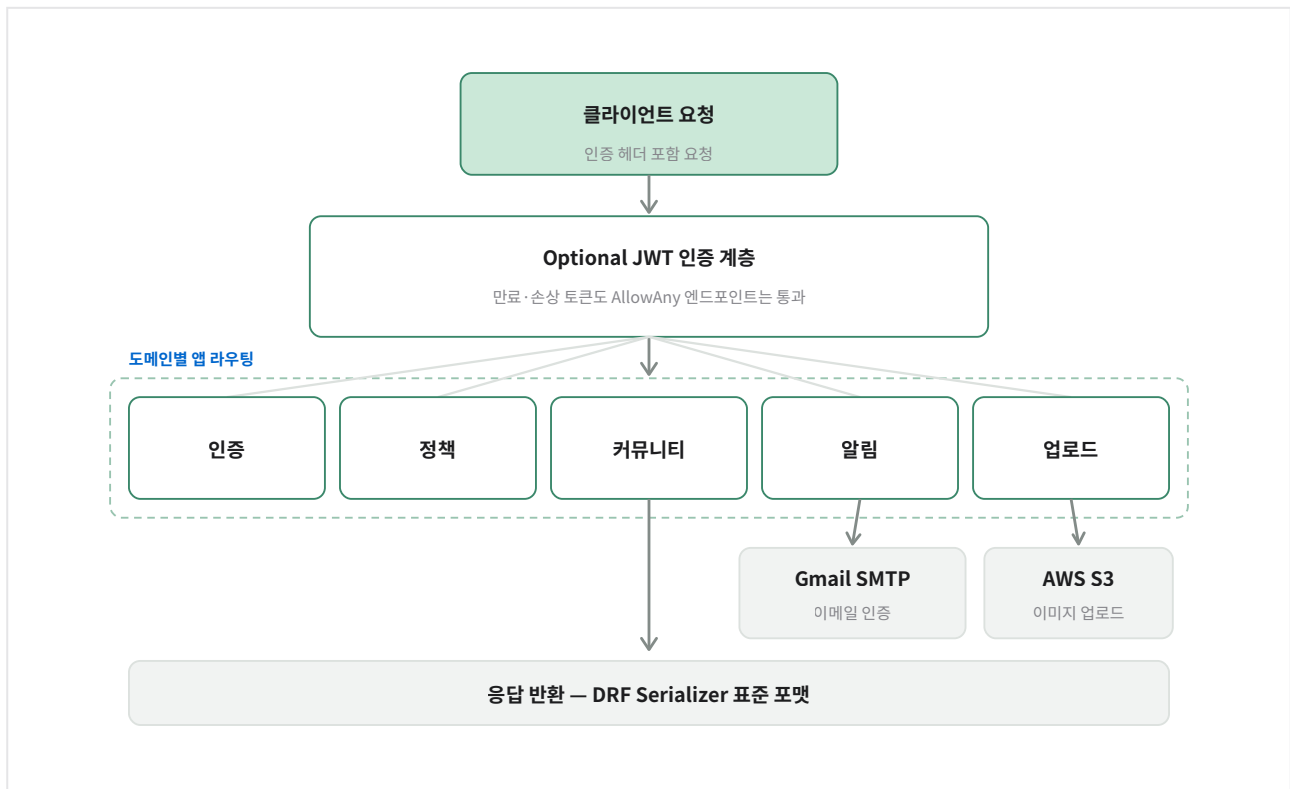
도메인 로직만으로는 서비스가 돌아가지 않습니다. 프로필 이미지는 어딘가 저장돼야 하고(S3), 회원가입은 실제 이메일로 검증돼야 하고(Gmail SMTP), 운영자는 데이터를 볼 화면이 있어야 합니다(Django Admin). 이 셋을 개별 태스크가 아니라 하나의 운영 기준으로 묶어 백엔드 레이어에 통합했습니다.

— Vector가 못 하는 질의를 Graph로 보완

ChromaDB의 의미 유사도 검색은 범주·관계 기반 질의에 취약합니다. Item → Domain 관계를 Neo4j에 저장해 Cypher로 보완하고, Vector 상위 5개와 Graph 신규 2개를 중복 제거 후 7개로 병합하는 파이프라인을 구현했습니다. Neo4j 연결 실패 시 Vector 파이프라인으로 자동 전환하는 fallback도 함께 넣었습니다.

설계와 결과

Phase 2 백엔드 요청 처리 흐름



Optional JWT 인증 계층을 통과한 요청이 도메인별 앱으로 라우팅되고, 알림은 Gmail SMTP·업로드는 AWS S3와 직접 연동됩니다. 전 도메인 응답은 DRF Serializer로 표준화해 반환합니다.

정량 성과

6개

도메인 앱 구조 Phase 1 단일 레이어 → Phase 2

Django REST Framework 기반으로 도메인 경계를 나눠 직접 설계한 구조입니다.

37/37

기능 테스트 전건 Pass 팀 전체 테스트 케이스

로컬·기능 단위 검증 기준, 이 중 인증 예외 케이스가 담당 영역의 핵심 검증 대상이었습니다. (테스트 계획·결과 보고서는 팀 QA 담당과 협업)

EC2

배포 환경 연동 AWS RDS + Docker

RDS PostgreSQL을 직접 구축하고 EC2에서 Django·React 연동을 검증했습니다. 캠프 종료 후 배포 서버는 현재 종료된 상태입니다.

기술 스택

Python · Django · Django REST Framework · Simple JWT · FastAPI · Streamlit · LangGraph · ChromaDB · Neo4j · OpenAI · PostgreSQL · AWS · Docker · Nginx

github.com/idenist/SKN29-4th-5TEAM

TrendIt

YouTube KR 트렌딩 지속성 예측 플랫폼

팀 5인 · 분석 설계 주도

0.8284

AUC-ROC

+13.6%

24h 반응 추가 효과

0.8475

3구간 분류 AUC

트렌딩에 오른 콘텐츠가 얼마나 오래 버티는지를, 진입 직후 정보만으로 예측할 수 있는가.

2022-2025년 YouTube KR 트렌딩 872K 스냅샷으로 콘텐츠의 트렌딩 지속 기간을 예측하는 ML 분석 플랫폼입니다. 단순 조회수·좋아요 피처를 넘어 카테고리별 소비 구조 차이와 초기 신호 속도를 반영했습니다.

담당 — 데이터 전처리와 클러스터링을 맡았고, T0/T0+24h 모델 분리 구조와 카테고리 보정 복합 타겟(TDI) 설계, 데이터 누수 방지 원칙 수립으로 팀의 분석 방향을 주도했습니다.

핵심 설계 판단

— 언제의 정보로 예측하는가를 먼저 나누다

진입 시점(T0) 정보만 쓰는 모델과 24시간 반응 데이터를 더한 모델을 완전히 분리했습니다. 두 파이프라인을 섞지 않았기 때문에 "하루를 기다리면 예측이 얼마나 좋아지는가"를 오염 없이 정량화할 수 있었습니다 — PR-AUC 0.582 → 0.661, +13.6%.

— 카테고리 편향을 흡수하는 복합 타겟 설계

Education은 오래 소비되고 Music은 짧게 소비됩니다. 지속 시간만으로 재면 카테고리 편향이 그대로 타겟에 들어옵니다. 지속 시간 × 순위 품질을 결합한 TDI 지수를 만들고, 임계 값 0.4를 클래스 비율(65:35)과 두 그룹 중간값 차이(약 6배)로 정량 검증한 뒤 확정했습니다.

— 누수를 설계 단계에서 차단하다

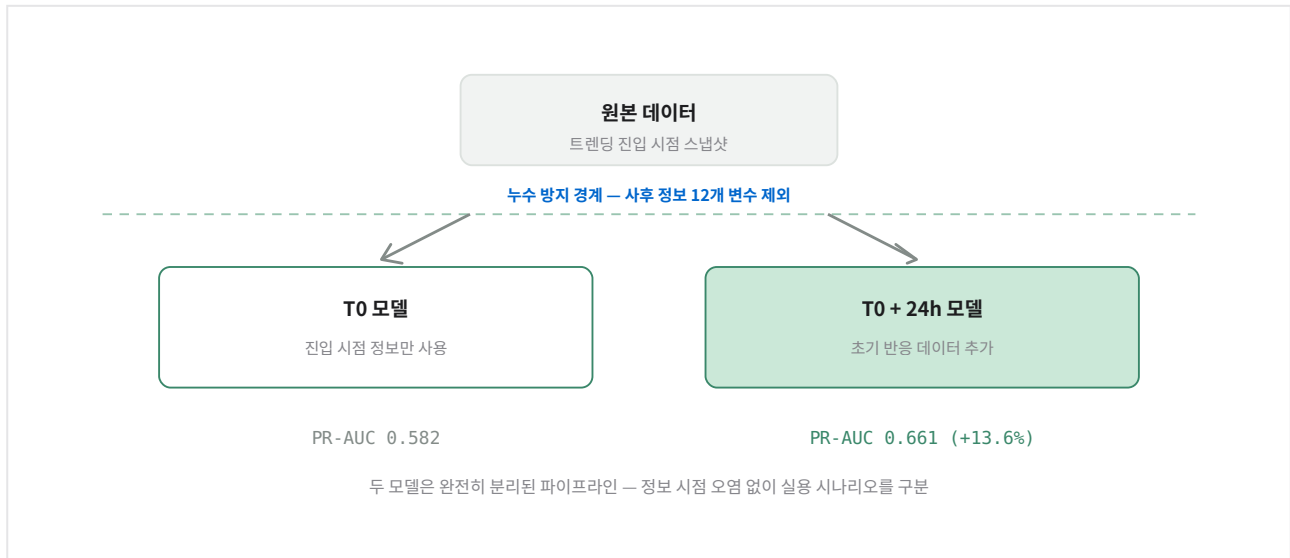
T0 이후에야 알 수 있는 사후 정보 12개 변수를 처음부터 제외했습니다. 클러스터링 과정에서 Y변수가 섞여 들어간 누수도 직접 발견해, cluster_id를 EDA 전용으로 격리하고 학습 파이프라인과 분리했습니다.

— EDA에서 발견한 것을 피처로 연결

latency_to_trend 분포를 뜯어보다 pretrend_view_velocity 피처를 제안했고, 시각화에서 드러난 시간대 주기성은 hour_sin/cos 인코딩으로 반영했습니다. EDA가 그림 그리기로 끝나지 않고 피처 설계와 성능 기여로 이어지도록 인과를 유지했습니다.

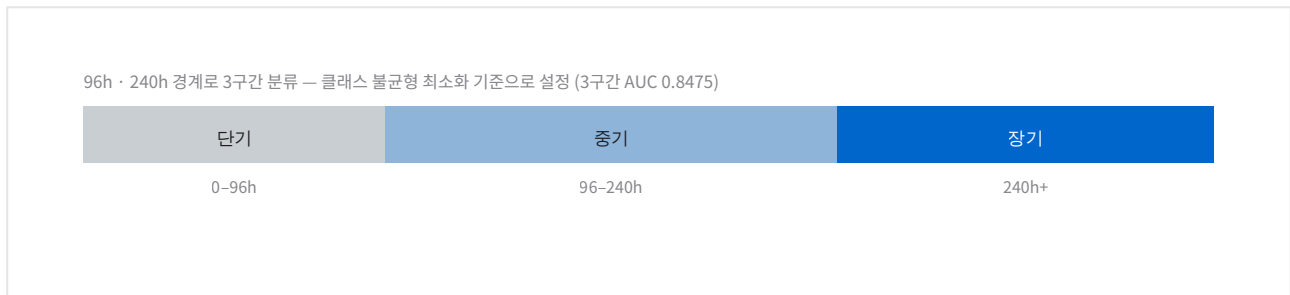
설계와 결과

누수 방지 구조 — T0 / T0+24h 파이프라인 분리



사후 정보 12개 변수를 설계 단계에서 제외해, T0 모델이 24시간 반응 데이터를 우연히라도 참조하지 못하도록 파이프라인 자체를 분리했습니다.

지속 구간 3분할



연속값 회귀는 R^2 0.5322로 분산의 절반만 설명했습니다. 나머지는 알고리즘 변화-뉴스 사이클 같은 통제 불가 요인이라, 연속 예측을 고집하는 대신 구간 분류로 접근을 바꿔 실용성을 확보했습니다.

정량 성과

0.8284

AUC-ROC 지속성 이진 판별

콘텐츠 2개를 비교했을 때 83% 확률로 더 오래 갈 쪽을 맞춥니다. 외부 변수가 많은 도메인에서 랜덤(0.50) 대비 실질적인 판별력입니다.

0.8475

3구간 분류 AUC Macro F1 0.657

XGBoost + LightGBM 앙상블 T0 최종 테스트 기준.

0.5322

Regression R^2 연속값 예측의 한계

분산의 53%만 설명 — 이 한계를 인정한 것이 구간 분류로 전환한 근거가 됐습니다.

기술 스택

Python · pandas · NumPy · XGBoost · LightGBM · CatBoost · PyTorch · K-Means · MySQL · Parquet · FastAPI · React

github.com/SKNETWORKS-FAMILY-AICAMP/SKN29-2nd-1Team

Forecasting Bitcoin and Ethereum Prices

A Markov Regime-Switching HAR Framework Based on Their Co-Movement · 학부 연구 논문

공동 연구 (지도교수) · 제1저자 · 모델링 실험 구현 주도

48개

전수 비교 모델

1.84%

BTC MAPE ($k=1$)

-10.8%

BTC MAPE 개선 ($k=10$)

두 자산이 함께 움직이는 국면을 관측 가능한 값으로 정의하면, 예측이 실제로 좋아지는가.

Bitcoin과 Ethereum의 가격 공동 예측을 위한 Markov Regime-Switching HAR 모델을 제안한 학부 연구 논문입니다. 2016.03~2024.11 일별 3,188건으로 AR·HAR 48개 모델을 전수 비교했고, Rolling-window 시변 상관계수를 기반으로 레짐을 직접 관측 가능하게 정의한 것이 핵심 기여입니다.

담당 — Introduction 전체와 Conclusion 집필, 48개 모델 비교 실험 코드 구현·실행·해석, 분석 결과 시각화(Figures 3~9) 구성.

레짐 정의·HAR 채택·임계값 선택은 지도교수와 공동 설계했고, 다단계(k -step) 예측 확장은 아이디어를 제안해 논의 후 반영했습니다. 누적 수익 G 공식은 지도교수 설계입니다.

핵심 설계 판단

— [공동 설계] 숨은 상태 대신 관측 가능한 상관계수로 레짐을 정의

Hidden Markov Model은 레짐이 잠재 변수라 해석이 어렵고 추정 복잡도가 높습니다. 대신 두 자산의 Rolling-window 시변 상관계수를 임계값과 비교해 레짐을 직접 정의했습니다. "지난 6개월 상관관계가 0.75 미만이면 독립 모델, 이상이면 공동 모델"로 읽히기 때문에 해석 가능성이 확보됩니다.

— [공동 설계] GARCH 대신 HAR — 여러 시간 스케일을 동시에

GARCH 계열은 단기 기억 한계로 암호화폐의 다중 시간 스케일 패턴을 놓칩니다. HAR은 1일·7일·15일 이동평균을 함께 써서 일간·주간·격주간 투자 행동을 반영하고, 차분이나 로그 변환 없이 원시가격에 적용해 수준 변수의 구조도 유지합니다(Bergsli et al. 2022의 선행 근거).

— [공동 설계] 임계값과 윈도우를 데이터로 고르다

임계값 2종 $\times$ 윈도우 2종의 조합에서 인샘플 MSE를 최소화하는 쌍을 탐색했습니다. HAR(3)의 경우 윈도우 182일이면 $\rho^*=0.75$, 365일이면 $\rho^*=0.64$ 가 선택돼 윈도우 크기에 따라 최적 임계값이 달라진다는 것을 실증했습니다. 전 구간 상관계수(0.9299)와 구간별 시변 상관계수의 괴리가 레짐 구분의 근거입니다.

— [단독 제안] 1단계 예측을 넘어 k 단계로, 수수료까지 반영

선행 연구는 1단계 예측만 다뤘습니다. $k=1,3,7,10$ 확장을 제안해 반영했고, 거래 수수료를 비용으로 넣은 누적 수익 기준으로 전략을 평가했습니다. 결론은 명확했습니다 — 1단계 전략은 수수료가 없을 때만 우위이고, 수수료가 있는 현실적 환경에서는 다단계 접근이 더 강건합니다.

— [직접 구현] 48개 조합을 전부 돌리고, 적합과 예측을 분리해 읽다

AR(1~4)·HAR(2~4) 각각에 4개 Markov 조합, 여기에 단일·이변량 기준 모델까지 48개를 직접 구현·실행했습니다. 적합 지표(RMSE·AIC·BIC)와 예측 지표(MAPE_k 등)를 분리 평가해 "적합이 좋다 $\neq$ 예측이 좋다"는 한계를 직접 해석하고, 최종 모델 선정 근거를 수치로 정당화했습니다.

설계와 결과

실험 설계 — 48개 모델의 적합도를 한눈에

48개 모델의 BTC 적합도(RMSE) — 진할수록 낮은 오차(더 좋은 적합)						
	단변량	이변량	M(0.5,ℓ*)	M(ρBE,ℓ*)	M(ρ*,182)	M(ρ*,365)
AR(1)	0.0472	0.0470	0.0451	0.0430	0.0435	0.0394
AR(2)	0.0472	0.0469	0.0449	0.0430	0.0420	0.0393
AR(3)	0.0471	0.0468	0.0452	0.0431	0.0420	0.0400
AR(4)	0.0472	0.0469	0.0449	0.0430	0.0419	0.0392
HAR(2) _{1,7}	0.0473	0.0470	0.0451	0.0430	0.0422	0.0394
HAR(2) _{1,15}	0.0473	0.0471	0.0451	0.0430	0.0422	0.0394
HAR(3) _{1,7,15}	0.0473	0.0472	0.0450	0.0430	0.0421	0.0393
HAR(4) _{1,7,15,30}	0.0474	0.0472	0.0453	0.0432	0.0435	0.0396

0.0392 적합 1위 — AR(4) · M(ρ*,365)
 0.0421 최종 선정 — HAR(3) · M(ρ*,182)

적합 1위와 최종 선정 모델이 다르다 — "적합이 좋다 ≠ 예측이 좋다"는 것을 이 논문이 구분하는 이유

48개 조합의 BTC 적합 오차(RMSE)를 색 농도로 표시했습니다. 가장 진한 칸(점선)은 AR(4)-M(ρ*,365)로 적합 기준 1위지만, 실제로 예측 성능까지 검증해 최종 선택한 모델은 HAR(3)-M(ρ*,182)(붉은 테두리)입니다. **적합 1위가 예측 1위가 아니었다**는 것을 48칸 전체를 놓고서야 확인할 수 있었습니다.

레짐에 따라 모델 구조 자체가 바뀐다

직전 182일 상관계수 ρ와 임계값 0.75의 비교로 레짐이 결정된다	
Regime 0 · 약상관 (ρ ≤ 0.75) BTC와 ETH를 각각 독립 모델로 예측 $a_0 \quad 0$ $0 \quad d_0$ 전체 기간의 36.5%	Regime 1 · 강상관 (ρ > 0.75) 두 자산을 하나의 공동 모델로 예측 $a_1 \quad b_1$ $c_1 \quad d_1$ 전체 기간의 63.5%
레짐 지속성 — 한 번 들어가면 잘 바뀌지 않는다 (p₀₀ = 0.992, p₁₁ = 0.995) 비대각 성분 b, c가 0이 되면 두 자산이 서로를 참조하지 않는다 — 구조 자체가 전환된다	

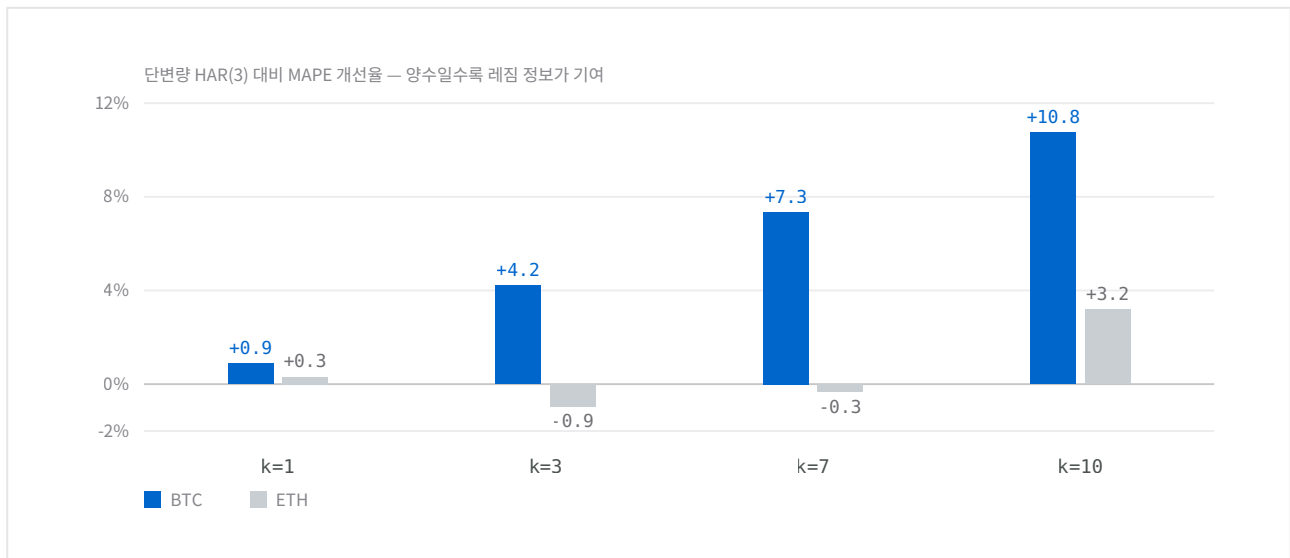
계수만 달라지는 것이 아니라 비대각 성분이 0이 되면서 두 자산이 서로를 참조하지 않게 됩니다. 즉 약상관 국면에서는 독립 모델 2개로, 강상관 국면에서는 공동 모델 1개로 전환됩니다. 레짐이 관측 가능한 값으로 정의돼 있어 "지금 어느 국면인지"를 그날의 상관계수만 보면 알 수 있습니다.

k단계별 MAPE — 레짐 정보는 어디서 도움이 되는가

	BTC MAPE (%)		ETH MAPE (%)	
	단변량 HAR(3)	M(ρ*,182)	단변량 HAR(3)	M(ρ*,182)
k=1	1.8592	1.8431	2.3631	2.3556
k=3	3.7669	3.6085	4.4834	4.5258
k=7	5.8514	5.4211	6.6322	6.6549
k=10	7.2048	6.4296	8.0000	7.7449

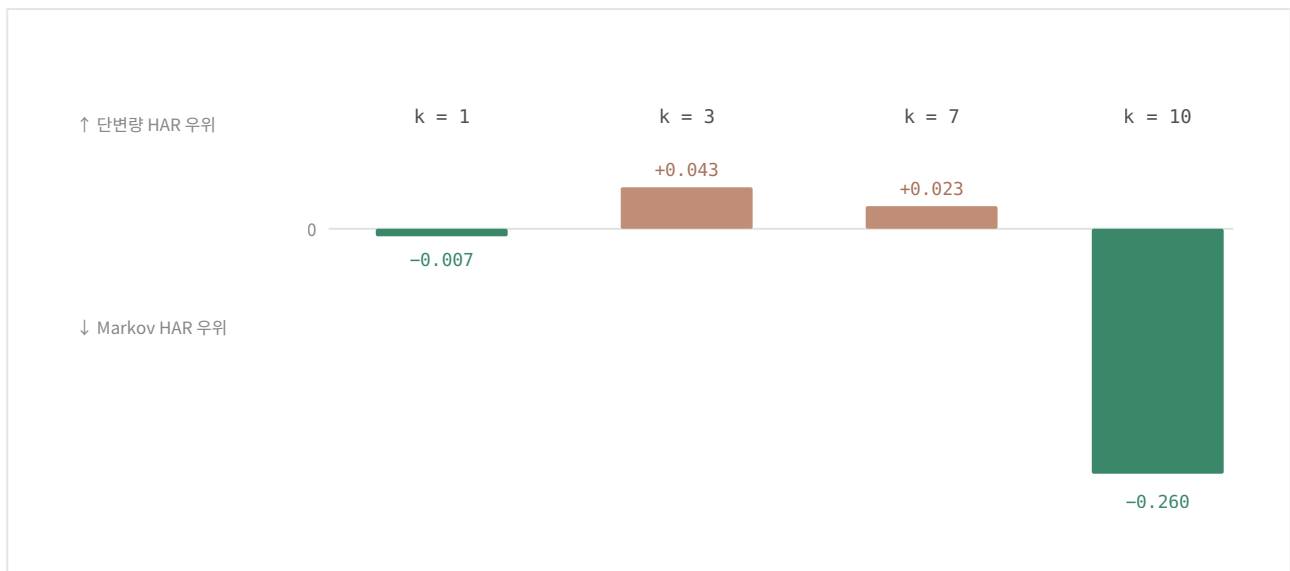
파란 값 = Markov 우위, 회색 값 = 단변량 역전. BTC는 k=1~10 전 구간에서 Markov가 우위, ETH는 중기(k=3~7)에서만 역전합니다. 아웃오브샘플 100일(m=100) 기준.

예측 구간이 길어질수록 레짐 정보의 값이 커진다



BTC는 개선율이 k=1에서 0.87%에 그치지만 k=10에서 10.76%까지 **단조 증가**합니다 — 멀리 볼수록 공동 구조 정보가 중요해진다는 뜻입니다. 반면 ETH는 중간에서 오히려 손해를 봤다가 k=10에서 회복하는 비단조 패턴을 보입니다. 같은 레짐 정보가 자산에 따라 다르게 작동한다는 것이 이 논문에서 가장 조심스럽게 해석해야 할 지점입니다.

레짐 정보의 기여 — ETH 구간별 비대칭



ETH의 MAPE 차이(Markov - 단변량)를 0 기준선으로 표시했습니다. 두 모델의 MAPE를 그대로 겹치면 차이가 0.02~0.04%p라 선이 포개져 판독이 불가능하기 때문입니다.

정량 성과

1.84%

BTC MAPE (k=1) 고정확도 기준 충족

Faghih et al.(2020)의 High Accuracy 기준(MAPE ≤ 10%)을 충족합니다.

-10.8%

BTC MAPE 개선 (k=10) 7.205% → 6.430%

BTC는 k=1부터 k=10까지 전 구간에서 Markov HAR이 단변량을 일관되게 능가합니다.

12.16%

AR 조합의 붕괴 지점 ETH k=7, ℓ=365 기준

AR(4)-M(p^* ,365)는 ETH 중장기에서 MAPE 12.16%·15.58%로 무너집니다. 윈도우를 6개월로 잡은 쪽이 1년보다 구조 변화 반응에서 안정적이라는 것을 보여주는 반례입니다.

Python · HAR Model · Markov Chain · Rolling Window Correlation · Least Squares Estimation · Time Series Analysis · RMSE / MAPE / RRSE · Prediction Intervals · Long-Short Strategy

github.com/kchan28/Forecasting-Bitcoin-Ethereum-Prices